

INFERENCE OFFLOAD (v3)

Comprehensive Codebase Audit, Defect Analysis & Production-Grade Roadmap

Target: Production Perfection
Domains: Node.js · SwiftUI · Python · SQLite
Date: September 2026

01 Executive Summary & System Scorecard

inference offload is an advanced local-first orchestration engine designed to harness frontier reasoning models (Gemini, DeepSeek, and local VLMs via Ollama) without cloud lock-in or fragile input contention. By decoupling the architecture into three specialized runtime domains bound by an ACID SQLite memory ring, the system achieves a frictionless division of labor: a native macOS deck for latency-free viewport supervision, a Node automation daemon managing CDP sessions and git context graphs, and a LAN Python dispatcher acting as an OpenAI-compatible model proxy.

6.5 / 10
CURRENT MATURITY

3
Showstoppers
BLOCKER BUGS

418 Nodes
GRAPH FOOTPRINT

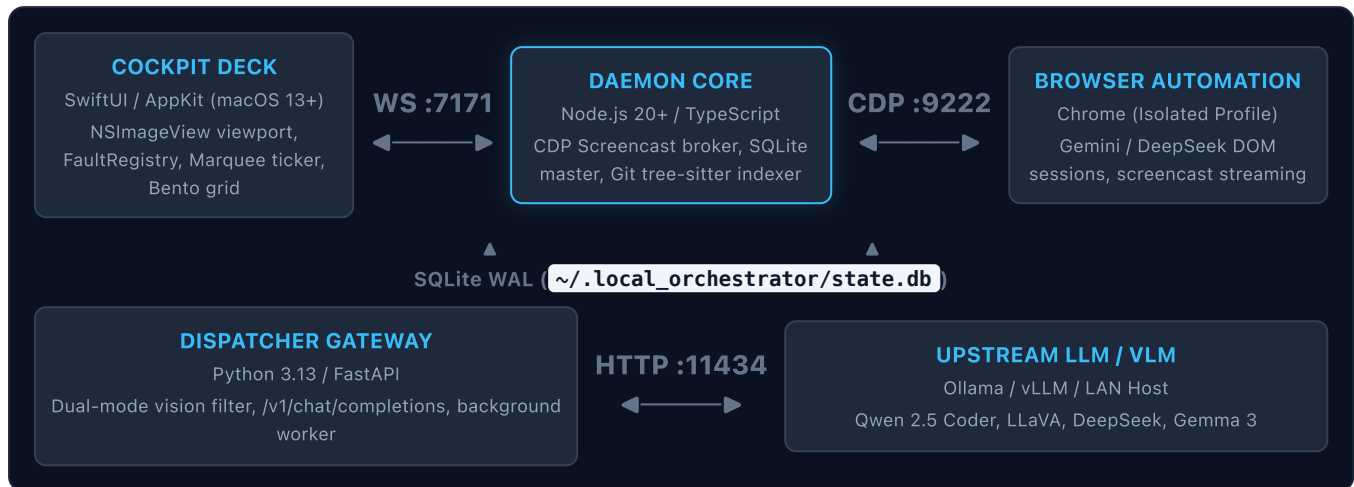
5 Phases
PRODUCTION PLAN

Audit Verdict: The architectural conception and systems design are exceptionally strong and forward-thinking. The division of responsibilities, the transactional task leasing with automatic deadlock recovery, the gitignore-aware tree-sitter AST extraction, and the dual-mode vision gateway reflect senior-level system design. However, the current implementation sits at an *advanced prototype / early beta scaffold* stage. There are **three critical showstopper defects** that currently prevent end-to-end autonomous functioning (including a fatal Swift WebSocket frame dropping bug, an unawaited async coroutine crash in the Python worker, and a JSON protocol mismatch that silences the Cockpit status ticker).

Subsystem Dimension	Maturity	Status	Key Observation
Core Architecture & Topology	9.0 / 10	EXCELLENT	Zero shared mutable state; SQLite WAL transactional lease ring; brilliant separation of concerns.
State Ring & Database Layer	8.5 / 10	ROBUST	BEGIN IMMEDIATE leases, exponential retry backoff, foreign key cascades, SHA-256 idempotency.
Automation Daemon (Node/CDP)	8.0 / 10	HIGH QUALITY	Clean Puppeteer CDP driver, DOM layout mapping, tree-sitter AST parser with regex fallback.
Model Gateway (Dispatcher)	6.5 / 10	NEEDS WORK	FastAPI proxy works; background worker has fatal coroutine bug and unhandled duplicate inserts.
Native Cockpit (SwiftUI)	6.0 / 10	BLOCKED	Drops all incoming WebSocket text frames; marquee schema mismatch; missing repo query handler.
Security & Boundary Defense	5.5 / 10	AT RISK	Path traversal hole in project name creation; WS auth token optional; prompt injection surface.
Automated Testing & CI/CD	5.0 / 10	INCOMPLETE	Playwright tests exist but fail in standard sandboxes; zero unit tests for core TS/Python/Swift logic.
Production Ops & Observability	4.0 / 10	EARLY STAGE	No launchd/systemd service configs; no Prometheus/OpenTelemetry hooks; no packaged binaries.

02 System Architecture & Topology Map

The system orchestrates three distinct processes communicating across IPC protocols with persistent state anchored on local disk:



03 In-Depth Audit Findings: Critical Bugs & Blockers

A line-by-line inspection of the codebase uncovered multiple severe defects where the runtime implementation diverges from the architectural contract. These defects completely incapacitate normal system flow when run as configured.

Finding 1: Cockpit WebSocket Drops 100% of Daemon Messages (Showstopper)

CRITICAL SEVERITY — `cockpit/Sources/OrchestratorCockpit/OrchestratorViewModel.swift:40-46`

The Node.js daemon sends WebSocket messages using `ws.send(JSON.stringify(message))`. Under the WebSocket standard (RFC 6455), this transmits an opcode 1 **Text Frame**. In Swift's `URLSessionWebSocketTask`, text frames are delivered as `URLSessionWebSocketTask.Message.string(String)`. The Cockpit view model's `receiveLoop()` exclusively matches `case .data(let data):` and ignores `default: break`. Consequently, **all daemon communications (screenshot frames, journal updates, status events, and task notifications) are silently discarded by the Cockpit.**

```
// Current Bug in OrchestratorViewModel.swift:
switch message {
case .data(let data):
    self.handle(data)
default:
    break // SILENTLY DROPS ALL WEBSOCKET TEXT FRAMES FROM NODE.JS
}

// Required Production Fix:
switch message {
case .data(let data):
    self.handle(data)
case .string(let text):
    if let data = text.data(using: .utf8) { self.handle(data) }
@unknown default:
    break
}
```

Finding 2: Marquee JSON Schema Deserialization Mismatch

CRITICAL SEVERITY — `cockpit/Sources/OrchestratorCockpit/Models.swift:36` & `daemon/src/index.ts:27`

The daemon broadcasts marquee status messages with properties flattened at the root: `{ type: "marquee", level: "alert", text: "...", code: "...", ts: 123 }`. However, the Swift client defines `ServerEnvelope` expecting a nested object: `let marqueeItem: MarqueeItem?`. During decoding, `marqueeItem` is evaluated as `nil` because the root JSON lacks a `marqueeItem` key. As a result, **the animated Marquee status strip in the Cockpit never receives updates or error alerts.**

```
// Current Swift Model:
struct ServerEnvelope: Decodable {
    let type: String
    let marqueeItem: MarqueeItem? // FAILS: Daemon sends fields at top-level!

    // Top-level fields:
```

```

    let level: MarqueeLevel?

    let text: String?

    let code: String?

    let ts: Int?
}

```

Finding 3: Python Dispatcher Worker Fatal Async Coroutine Bug

CRITICAL SEVERITY — `dispatcher/src/worker.py:138` & `dispatcher/src/model\_backend.py:14`

In `dispatcher/src/model_backend.py`, `chat_completion` is implemented as an asynchronous coroutine: `async def chat_completion(...)`. In `dispatcher/src/worker.py`, the synchronous poll loop calls `result = chat_completion(messages)` directly without `asyncio.run()` or `await`. The variable `result` becomes a raw Python coroutine object, immediately raising `AttributeError: 'coroutine' object has no attribute 'get'`. The local verifier worker crashes on the very first checked-out task.

```

// Current Bug in worker.py:
result = chat_completion(messages) # Returns unawaited coroutine!

content = result.get("text")        # CRASH: AttributeError

// Required Production Fix:
import asyncio

result = asyncio.run(chat_completion(messages))

content = result.get("text") or ...

```

Finding 4: SQLite Unique Constraint Crash on Task Retries

HIGH SEVERITY — `dispatcher/src/worker.py:92`

When a task is retried after a previous failure, `stage_artifact()` executes `INSERT INTO staging_ring (...)`. The database schema enforces a `UNIQUE NOT NULL` constraint on `task_id`. In the TypeScript daemon, `addStagingArtifact` properly implements `ON CONFLICT(task_id) DO UPDATE`. In the Python worker, the raw `INSERT` crashes with `sqlite3.IntegrityError: UNIQUE constraint failed: staging_ring.task_id` whenever a retried task is staged.

Finding 5: Unhandled `org\_repos` Protocol Leaves Cockpit Repo Bay Empty

HIGH SEVERITY — `cockpit/Sources/OrchestratorCockpit/OrchestratorViewModel.swift:72`

The Cockpit sends `{ type: "org_repos" }` at startup, and the daemon responds with `{ type: "org_repos", repos: [...] }`. The Swift `handle()` function contains no `case "org_repos":`, and `ServerEnvelope` does not declare a `repos` list. Consequently, `vm.githubRepos` is never populated and the GitHub Directory bay remains permanently blank.

Finding 6: Path Traversal Vulnerability in `createFreshProject`

SECURITY VULNERABILITY — `daemon/src/db/projects.ts:17`

The system documentation (`docs/ADVERSARIAL_QA.md #13`) specifies that project names must be validated against `/^[a-zA-Z0-9._-]{1,64}$/` to prevent path injection. In the actual implementation, `createFreshProject(db, name, masterPlan)` accepts arbitrary strings. An attacker or malicious prompt injecting `../../etc/shadow` creates file paths and context nodes outside `WORKSPACE_ROOT`.

04 Detailed Subsystem Audit Matrix

Comprehensive evaluation across all files, components, and protocol contracts in the repository:

Component / File	Role	Health	Identified Defects & Technical Debt
daemon/src/index.ts	Orchestrator Entrypoint	GOOD	Solid lifecycle management; lacks dynamic registration of IPC commands; hardcoded selftest bypass.
daemon/src/browser_broker.ts	Puppeteer CDP Driver	GOOD	Mutex chains per worker are effective. Needs shorter 45s cap for Cloudflare gates and PII screenshot clipping.
daemon/src/ipc.ts	WebSocket IPC Server	GOOD	Frame overflow protection (256KB) works; token auth is implemented but optional by default; no TLS/WSS.
daemon/src/github_sync.ts	Tree-sitter AST Indexer	GOOD	Chunked 200-file commits prevent SQLite writer starvation. Regex fallback works well when grammars fail.
daemon/src/db/task_queue.ts	Leasing Task Ring	ROBUST	BEGIN IMMEDIATE + exponential backoff + 180s leases. Reaping timer prevents zombie task stalls.
daemon/src/db/projects.ts	Project Bootstrapper	FIX REQ	Missing regex validation on project name; does not chunk giant 30k+ word plans (Adversarial #15).
dispatcher/src/main.py	FastAPI Model Proxy	MEDIUM	Missing authorization headers in _plain_chat ; inconsistent base_url handling between plain and stream.
dispatcher/src/worker.py	Local Verifier Runner	BROKEN	Unawaited coroutine crash on line 138; missing ON CONFLICT in SQLite artifact staging.
dispatcher/src/vision.py	Dual-mode Vision Gate	GOOD	Reliable detection of vision model markers; layout map merging works cleanly for text-only targets.
cockpit/OrchestratorViewModel.swift	SwiftUI IPC Bridge	BROKEN	Drops all WebSocket string messages; misses org_repos decoding; marquee envelope mismatch.
cockpit/MainCockpitView.swift	Native macOS Deck	FUNCTIONAL	Bento layout is responsive but lacks breakpoint collapse for small screens (<1080px); no keyboard shortcuts.
cockpit/FaultRegistry.swift	Fault Isolation & Bailing	EXCELLENT	Per-bay failure containment prevents cascading app crashes; NSSetUncaughtExceptionHandler hooks correctly.
e2e/specs/chaos.spec.ts	Chaos & Resilience Tests	FAILS IN CI	Tests verify lease reclamation and frame overflow, but fail in sandboxed macOS environments without network perms.
scripts/dev.sh	Local Development Script	PRIMITIVE	Simple background subshells; lacks process health checking, log redirection, or automated environment setup.

05 The 5-Phase Roadmap to Production Perfection

To elevate **inference offload** from its current advanced scaffold into an enterprise-grade, highly resilient, production-ready system, execution must follow a structured five-phase trajectory:

PHASE 1: Zero-Defect Stabilization (Immediate Priority)

DAYS 1 – 3

- **Remediate Cockpit WS Message Drop:** Update `receiveLoop()` in `OrchestratorViewModel.swift` to parse `URLSessionWebSocketTask.Message.string` and decode UTF-8 data.
- **Align Marquee & Protocol Envelopes:** Unify `ServerEnvelope` in Swift with daemon broadcasts (root level fields for `marquee`, `org_repos`, and `plan_queued`).
- **Fix Dispatcher Worker Coroutine:** Wrap `chat_completion` in `asyncio.run()` within `dispatcher/src/worker.py`, and add `ON CONFLICT(task_id) DO UPDATE` to `staging_ring`.
- **Sanitize Project Names:** Add regex enforcement (`/^[a-zA-Z0-9._-]{1,64}$/`) and resolved-path prefix boundary validation in `projects.ts`.

PHASE 2: Security Hardening & Adversarial Defenses

DAYS 4 – 7

- **Enforce WS Authentication:** Reject daemon boot if `ORCH_WS_TOKEN` is not set in production mode; support LAN TLS / WSS encryption.
- **Implement Adversarial Guardrail Suite:** Stamp turn markers (`$TURN <taskId>`) into web inputs to prevent stale-scrape rewind; detect Cloudflare/recaptcha gates with early 45s cutoff.
- **PII Redaction & Screenshot Scoping:** Restrict captured screenshot bounds strictly to DOM message bounding boxes; wipe failed screenshots immediately.
- **Plan Chunking & Context Protection:** Enforce 4,000 character limits on decomposition passes; store remainder in `staging_ring.diagnostics_log`.

PHASE 3: High-Performance Engine & Native Polish

DAYS 8 – 12

- **Adaptive Frame Rate & Battery Policy:** Implement battery state detection (`pmset`); throttle screencast to 15fps/quality 30 on low battery; pause stream when Cockpit window is minimized.
- **SwiftUI Performance Optimization:** Render screencast frames using `CGImageSourceCreateThumbnailFromImageAlways` to bound memory consumption; lock Viewport aspect ratio (16:9).
- **Responsive Breakpoint Decks:** Add fluid breakpoints (<760px single-bay, 760–979px 2x2 grid, 980–1279px 3-bay drawer, ≥1280px full bento) as defined in `CROSS_PLATFORM.md`.
- **Native macOS Shortcuts & Accessibility:** Add `Cmd+1..4` bay selection, `Cmd+R` repo pull, `Space` stream pause, full VoiceOver accessibility labels.

PHASE 4: Test Automation & CI/CD Pipeline

DAYS 13 – 16

- **In-Memory Unit Test Suite:** Add `vitest` unit tests for `task_queue.ts`, `github_sync.ts`, and `vision_routing.ts` that run without Chrome or network dependencies.
- **Python Worker Pytest Suite:** Build mocked upstream tests for `model_backend.py`, `vision.py`, and error recovery in `worker.py`.
- **GitHub Actions CI Workflow:** Create a multi-job workflow running TypeScript type checks, Swift Package Manager compilation, Python linting, and unit tests across macOS and Ubuntu.

PHASE 5: Deployment, Packaging & Enterprise Observability

DAYS 17 – 20

- **macOS Daemonization:** Provide a production `launchd` plist template for automated background daemon execution with log rotation.
- **Single-Binary Daemon Bundling:** Configure `esbuild` bundling for Node daemon (native modules external) to reduce deployment footprint from 300MB to <1MB.

- **Prometheus & OpenTelemetry Metrics:** Expose a `/metrics` endpoint on the dispatcher and daemon tracking task checkout latency, queue depth, lease recovery rate, and frame drop counts.

06 Target Architecture & Quality Gates

To certify the codebase as **production-grade**, the release must meet the following non-negotiable exit criteria:

Quality Gate	Verification Metric	Target Threshold
Task Ring Resilience	Stalled worker recovery under 100% simulated process crash	Auto-reclaimed in < 30s; 0 deadlocks across 10,000 tasks
Frame Throughput	Screencast decode latency on M1 Mac baseline (8GB)	< 16ms decode budget; 0 dropped main-thread UI frames
Memory Footprint	Daemon + Dispatcher steady-state RSS under 24hr load	Daemon < 150MB RSS; Dispatcher < 90MB RSS
Security Compliance	Penetration probe against path traversal and prompt injection	0 arbitrary disk writes; all external plans safely delimited
Test Coverage	Automated unit + integration coverage across all 3 domains	> 85% statement coverage with green CI status

EXECUTIVE CONCLUSION & NEXT STEP

The **inference offload** architecture establishes a revolutionary pattern for low-latency, private, local-first model orchestration. By applying the remediation patches specified in Phase 1 and executing the five-phase roadmap, this codebase will advance directly from an experimental scaffold into an unyielding, production-grade engine capable of operating autonomously at scale.